

Proyecto 1

Ángel Samuel Alcántara
Acosta

Geiber Humberto Yañez
Castañeda

01/09/2026

Retrospectiva

¿Cuáles fueron los mini-ciclos definidos? Justifiquenlos.

- **Mini-ciclo 1: Modelado base y componentes gráficos.** Nos enfocamos en construir el esqueleto de las clases Symbol, Wheel y la inicialización de SlotMachine con sus 3 ruedas por defecto. Se justificó para garantizar que los elementos gráficos básicos funcionaran antes de meterle lógica.
- **Mini-ciclo 2: Operaciones dinámicas de la máquina.** Implementamos la capacidad de agregar y quitar ruedas (addWheel, delWheel) y gestionar símbolos (addSymbol, delSymbol), reajustando posiciones en pantalla. Se justificó porque la máquina debía ser flexible y no quedarse estática con solo 3 ruedas.
- **Mini-ciclo 3: Lógica de juego, estados y pruebas unitarias.** Desarrollamos la mecánica de giro (spin), la verificación de premios (isJackpot), el indicador de éxito (ok()) y los tests con JUnit (Test_slotMachine). Se justificó para asegurar la correcta ejecución del sistema y cumplir con la calidad del código.
- **¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?**

Actualmente nos encontramos finalizando el **Mini-ciclo 3**. Esto se debe a que ya logramos implementar el código de las tres clases principales, la lógica completa de giros/jackpot y realizamos la suite de pruebas unitarias (Test_slotMachine) para validar que cada método responda de forma correcta ante datos válidos e inválidos.

- **¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)**
- **Angel Samuel Alcántara Acosta:** 10 horas
- **Geiber Humberto Yañez Castañeda:** 10 horas
- **¿Cuál consideran fue el mayor logro? ¿Por qué?**

Nuestro mayor logro fue conectar de forma limpia la lógica orientada a objetos con el manejo dinámico de la interfaz gráfica en BlueJ. Lograr que al agregar o eliminar una rueda (addWheel/delWheel) el marco azul reajustara su tamaño y

las ruedas recalcularan sus posiciones X automáticamente sin romper el estado interno fue un gran paso.

- **¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?**

El mayor problema técnico fue sincronizar la lista interna `ArrayList<Wheel>` con el reordenamiento visual de cada elemento en el Canvas, además del manejo de índices fuera de rango. Lo resolvimos aplicando un método auxiliar `limit()` para controlar los límites y ajustando las posiciones dentro de un bucle tras cada inserción o eliminación. También tuvimos el inconveniente de que la licencia de Astah se nos venció en pleno trabajo y no pudimos renovarla, por lo que tuvimos que resolver el diseño de diagramas apoyándonos en herramientas alternativas.

- **¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?**

Destacamos el trabajo colaborativo constante y la repartición equitativa de las tareas. No nos dividimos el código a ciegas; siempre estuvimos al tanto de lo que escribía el otro para evitar incoherencias entre clases. Nos comprometemos a mejorar la planeación de horarios para diagramación previa, evitando contratiempos de último minuto con herramientas externas.

- **Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿por qué?**

La práctica más útil fue **Programación en Parejas (*Pair Programming*)** combinada con **Desarrollo Guiado por Pruebas (*TDD*)**. Mientras uno escribía la lógica en BlueJ (por ejemplo, corregir la iteración de `deleteSymbols` o la lógica de `isJackpot`), el otro iba detectando fallos de sintaxis e índices en tiempo real. Además, ir construyendo la clase `Test_slotMachine` nos permitió detectar rápido cuándo un método rompía la bandera ok.

- **¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.**

Nos apoyamos en la documentación oficial de la API de Java para el manejo de colecciones (`ArrayList`), las guías del curso para el uso de la librería gráfica

Canvas/Rectangle y en Inteligencia Artificial como asistente de sintaxis y depuración de pruebas unitarias.

- Google. (2026). *Gemini* (versión de septiembre de 2026) [Modelo de lenguaje grande]. <https://gemini.google.com>